JOHANNES KEPLER
UNIVERSITY LINZ

Author
Sandro Tobias Müller, BSc
52010874

Submission
Institute of Machine Learning

Thesis Supervisor
Dr. **Markus Holzleitner**

August 27, 2026

Practical Work in AI(MSc)

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract

This report documents the practical part of a seminar in AI centered on *Optimal Deep Learning of Holomorphic Operators Between Banach Spaces* by Adcock, Dexter and Moraga [1] (arXiv:2406.13928). We reproduce two of the paper's three numerical experiments – the parametric elliptic diffusion equation (a Hilbert-valued operator) and the Navier–Stokes–Brinkman equations (a Banach-valued operator) – end to end, rather than with a simplified proxy: mixed finite element data generation in FEniCS matching the paper's variational formulations, deterministic Clenshaw–Curtis sparse-grid test-error evaluation, and the paper's stated training protocol (architecture family, initialization, optimizer, and checkpoint/restore rule). We implement and train both experiments in PyTorch and JAX on identical data, giving a controlled comparison of the two frameworks in addition to the comparison against the original paper's reported results. Every point at which our implementation necessarily deviates from an underspecified or ambiguous detail in the paper – for example the exact finite element family used for the Navier–Stokes–Brinkman system, which is not stated in the paper itself – is documented explicitly.

Kurzfassung

Dieser Bericht dokumentiert den praktischen Teil eines KI-Seminars zum Thema *Optimal Deep Learning of Holomorphic Operators Between Banach Spaces* von Adcock, Dexter und Moraga [1] (arXiv:2406.13928). Wir reproduzieren zwei der drei numerischen Experimente des Papers – die parametrische elliptische Diffusionsgleichung (ein Hilbert-wertiger Operator) und die Navier–Stokes–Brinkman-Gleichungen (ein Banach-wertiger Operator) – vollständig und nicht anhand einer vereinfachten Approximation: Die Trainingsdaten werden mittels gemischter Finite-Elemente-Methoden in FEniCS gemäß den Variationsformulierungen des Papers erzeugt, der Testfehler wird über eine deterministische Clenshaw–Curtis-Dünngitter-Quadratur ausgewertet, und das im Paper beschriebene Trainingsprotokoll (Architekturfamilie, Initialisierung, Optimierer und Checkpoint/Restore-Regel) wird eingehalten. Beide Experimente werden sowohl in PyTorch als auch in JAX auf identischen Daten trainiert, was einen kontrollierten Vergleich der beiden Frameworks zusätzlich zum Vergleich mit den ursprünglich berichteten Ergebnissen ermöglicht. Jede Stelle, an der unsere Implementierung notwendigerweise von einem im Paper nicht eindeutig spezifizierten Detail abweicht – etwa die genaue Finite-Elemente-Familie für das Navier–Stokes–Brinkman-System, die im Originalpaper nicht angegeben ist – wird explizit dokumentiert.

Contents

Chapter 1

Introduction

Partial differential equations (PDEs) are central tools in scientific computing, where they are used to model physical systems such as diffusion, fluid flow, heat transport, and wave propagation. In many applications, a PDE has to be solved repeatedly for different input parameters, coefficient fields, source terms, initial conditions, or boundary conditions. This repeated-solve setting can be computationally expensive when classical numerical methods, such as finite element or finite difference methods, are used directly.

Operator learning addresses this problem by constructing surrogate models for PDE solution maps. Instead of solving the PDE from scratch for every new input, one trains a model that approximates the map from input data to the corresponding solution field. Neural-network-based operator learning is therefore a promising approach for accelerating parametric PDE simulations, especially in settings where many evaluations of the solution operator are required.

This project is based on the paper *Optimal Deep Learning of Holomorphic Operators Between Banach Spaces* by Adcock, Dexter and Moraga [1]. The paper studies deep learning methods for holomorphic operators between Banach spaces and provides near-optimal generalization guarantees for such operators. It is particularly relevant because it connects abstract operator learning theory with numerical experiments on parametric PDEs.

The aim of this project is to reproduce two of the paper's three numerical experiments – the parametric elliptic diffusion equation and the Navier–Stokes–Brinkman (NSB) equations – as faithfully as the available time and hardware permit, rather than with a simplified proxy. Concretely, this means implementing the paper's actual mixed finite element data generation (in FEniCS), its deterministic sparse-grid test-error protocol, and its stated training procedure, instead of substituting a finite-difference solver or an i.i.d. random test set. The diffusion experiment is Hilbert-valued (the solution is measured in an $L^2(\Omega)$-type norm), while the NSB experiment is Banach-valued (its mixed formulation produces a velocity field in $\mathbf{L}^4(\Omega)$), so together the two reproduced experiments span both regimes the paper's theory addresses. The corresponding parameter-to-solution maps are learned using fully connected neural networks implemented in both PyTorch and JAX. The reproduction investigates how the test error depends on the number of training samples, the parametric dimension, the activation function, and the implementation framework, and additionally compares the two frameworks against each other directly.

The remainder of this report is structured as follows. Section 2 introduces the relevant background on operator learning, Banach and Hilbert spaces, and the main contributions of the target paper. Section 3 states the concrete objectives

of this practical work, and Section 4 summarizes the parts of the paper's theory most relevant to the reproduction. Section 5 describes the parametric PDE problems, their mixed finite element formulations, and the learning formulation. Sections 6 and 7 present the PyTorch and JAX implementations, respectively, including the framework comparison. Section 8 reports the numerical results. Finally, Section 9 summarizes the findings and discusses limitations of the reproduction.

Chapter 2

Background

2.1 Operator Learning

Operator learning studies the approximation of mappings between function spaces. In the setting of PDEs, the object of interest is often a solution operator

$$F : \mathcal{X} \to \mathcal{Y}, \qquad X \mapsto F(X),$$

where $\mathcal{X}$ denotes an input space and $\mathcal{Y}$ denotes an output or solution space. The input X may represent a coefficient field, forcing term, initial condition, boundary condition, or another parameter-dependent quantity, while $F(X)$ denotes the corresponding PDE solution.

The idea of approximating non-linear operators with neural networks goes back at least to the universal approximation theorem for operators by Chen and Chen [6]. More recent neural operator methods include DeepONet [16], the Fourier Neural Operator [15], and the general neural operator framework of Kovachki et al. [13]. These methods differ in architecture, but share the goal of learning mappings between function spaces rather than only finite-dimensional vector-valued functions.

In the target paper, the learned operator is represented in an encoder–network–decoder form,

$$\mathfrak{p} = D_{\mathcal{Y}} \circ \mathfrak{N} \circ E_{\mathcal{X}},$$

where

$$E_{\mathcal{X}} : \mathcal{X} \to \mathbb{R}^{d_x}, \qquad \mathfrak{N} : \mathbb{R}^{d_x} \to \mathbb{R}^{d_y}, \qquad D_{\mathcal{Y}} : \mathbb{R}^{d_y} \to \mathcal{Y}.$$

Here, $E_{\mathcal{X}}$ maps the input object into a finite-dimensional representation, $\mathfrak{N}$ learns the finite-dimensional mapping, and $D_{\mathcal{Y}}$ reconstructs an approximation in the output space [1].

The training data are assumed to have the form

$$\{(X_i, F(X_i) + E_i)\}_{i=1}^{m},$$

where X_i are sampled from a probability measure μ on the input space and E_i represents noise or numerical error. The learning objective is to obtain an operator approximation that generalizes well from a finite number of samples. This is particularly important for PDE applications, where generating each training sample may require an expensive numerical solve.

For the practical reproduction in this project, the infinite-dimensional problem is reduced to a finite-dimensional supervised learning problem. A parameter vector

$$x \in [-1, 1]^d$$

is sampled, a numerical PDE solver generates the corresponding solution $u(x)$, and a neural network is trained to approximate the parameter-to-solution map

$$x \mapsto u(x).$$

Thus, the implementation focuses on the finite-dimensional surrogate learning problem induced by the original operator learning formulation.

2.2 Banach vs. Hilbert Setting

A key distinction in the target paper is the difference between Hilbert-valued and Banach-valued operator learning. A Hilbert space is a complete inner-product space. Its norm is induced by an inner product, which provides geometric tools such as orthogonality, projections, Parseval identities, and spectral decompositions. The space $L^2(\Omega)$ is the standard example of a Hilbert space in PDE analysis.

A Banach space is a complete normed vector space, but it does not necessarily have an inner product. Every Hilbert space is a Banach space, but not every Banach space is Hilbert. For example, $L^p(\Omega)$ is a Banach space for $1 \leqslant p \leqslant \infty$, but it is Hilbert only when $p = 2$. This distinction is important because many analytical tools available in Hilbert spaces are no longer available in general Banach spaces.

Much of the theoretical work on operator learning is formulated in Hilbert-space settings, which is natural for PDEs whose solutions are measured in L^2-based norms. However, more general PDE formulations, including mixed formulations and non-linear systems, may naturally involve Banach spaces such as $L^4(\Omega)$ or spaces involving L^q divergence conditions. The target paper therefore studies operators whose outputs may lie in general Banach spaces [1].

This broader setting creates additional analytical difficulties. In the Hilbert case, errors can often be controlled using the standard Bochner $L^2_\mu(\mathcal{X}; \mathcal{Y})$ norm. In the Banach case, the paper also uses the Pettis norm, which measures the operator error after testing against bounded linear functionals from the dual space $\mathcal{Y}^*$. This is weaker than the Bochner norm in general, but it is more suitable for Banach-valued analysis.

The theoretical bounds in the paper are weaker in the Banach case than in the Hilbert case. This difference is partly due to the lack of an inner-product structure and the absence of Hilbert-space tools such as Parseval's identity. However, the numerical experiments suggest that this theoretical gap may partly come from the proof technique rather than from a fundamental limitation of deep learning methods.

This project reproduces two of the paper's numerical experiments, chosen specifically because they instantiate both settings described above. The parametric elliptic diffusion equation is the simplest example in the target paper and corresponds to a Hilbert-valued output setting, since the solution is measured in an $L^2(\Omega)$-type space. The Navier–Stokes–Brinkman equations, in contrast, are solved through the mixed formulation of Gatica, Núñez and Ruiz-Baier [10], whose primary unknown velocity field lies in $\mathbf{L}^4(\Omega)$, a genuinely

Banach-valued setting in the sense discussed above. Reproducing both experiments therefore lets this report demonstrate, rather than merely describe, the practical consequence of the Hilbert/Banach distinction: identical network architectures and training protocol are applied to an L^2-valued and an L^4-valued operator, and the resulting test-error behavior can be compared directly.

2.3 Relevance of the Target Paper

The paper by Adcock, Dexter and Moraga [1] is the central reference for this practical work because it combines a theoretical analysis of deep operator learning with numerical experiments on parametric PDEs. The authors study the problem of learning operators of the form

$$F : \mathcal{X} \to \mathcal{Y},$$

where the input and output spaces may be infinite-dimensional function spaces. This setting matches the structure of many PDE problems, where an input coefficient, forcing term, or parameter field is mapped to the corresponding solution field.

A particularly relevant aspect of the paper is its focus on holomorphic operators. In parametric PDEs, the solution map often depends smoothly or analytically on the input parameters under suitable assumptions. This additional regularity can be exploited to obtain stronger approximation and learning guarantees than would be possible for general Lipschitz operators. The target paper uses this structure to prove near-optimal generalization bounds for deep learning methods applied to holomorphic operators.

The paper is also relevant because it goes beyond the standard Hilbert-space setting. Much of the existing theory for operator learning is formulated for operators whose outputs lie in Hilbert spaces, such as $L^2(\Omega)$. However, some PDE formulations naturally lead to Banach-valued solution spaces, for example spaces such as $L^4(\Omega)$. By considering operators between Banach spaces, the paper addresses a more general setting that is closer to certain mixed and nonlinear PDE formulations.

For the practical part of this project, the most important part of the paper is the numerical study. The authors test fully connected neural networks on several parametric PDE problems, including an elliptic diffusion equation, the Navier-Stokes-Brinkman equations, and a stationary Boussinesq equation. These experiments provide a clear reproduction target: generate parametric PDE data, train a neural network surrogate, and study how the test error changes with the number of training samples.

This project reproduces the parametric elliptic diffusion equation and the Navier–Stokes–Brinkman equations, the two Hilbert- and Banach-valued examples from the paper's numerical study (the third, the stationary Boussinesq equation, is outside the scope of this practical work). Both experiments are reproduced with the paper's own experimental pipeline rather than a simplified proxy: the training and test data are generated by solving the mixed finite element formulations stated in the paper's appendix, the test error is evaluated on a deterministic Clenshaw–Curtis sparse-grid quadrature exactly as the paper

specifies, and fully connected neural networks are trained following the paper's stated architecture and optimization protocol. In particular, the project follows the same general idea of sampling parameter vectors, computing corresponding PDE solutions, training fully connected neural networks, and evaluating the relative test error, while additionally comparing PyTorch and JAX implementations of the full pipeline against each other.

The target paper therefore serves two roles in this work. First, it provides the mathematical motivation for studying neural-network approximations of parametric PDE solution operators. Second, it provides the experimental reference against which the PyTorch and JAX reimplementations in this project are compared.

Chapter 3

Problem Statement and Objectives of this Practical Work

The target paper provides both theoretical guarantees and numerical experiments for deep learning of holomorphic operators between Banach spaces. The practical part of this report reproduces two of the paper's three numerical experiments as faithfully as the available time and hardware permit, using the paper's own data-generation pipeline rather than a simplified proxy, and implements the resulting learning problem in two modern machine learning frameworks.

The selected reproduction problems are the parametric elliptic diffusion equation and the Navier–Stokes–Brinkman (NSB) equations. The diffusion equation is the simplest numerical example considered in the target paper and gives a Hilbert-valued operator, while the NSB equations are solved through a mixed finite element formulation whose primary unknown is Banach-valued; together they cover both regimes addressed by the paper's theory (Section 2). The stationary Boussinesq equation, the paper's third experiment, is out of scope for this practical work. The main task is to generate parameter-dependent PDE data using the paper's stated finite element discretizations and sparse-grid test protocol, train neural network surrogates following the paper's training procedure, and compare the observed learning behaviour with the qualitative results reported in the paper.

3.1 Reproduction Objective

The first objective is to reproduce the general structure of the numerical experiments from the target paper. In the original work, the authors sample parameter vectors, solve a corresponding parametric PDE, train fully connected neural networks on the resulting input-output pairs, and evaluate the relative test error as the number of training samples increases.

In this project, the same workflow is followed for the diffusion and NSB problems, using the paper's own finite element discretizations rather than a simplified stand-in. Given a parameter vector

$$x \in [-1, 1]^d,$$

a mixed finite element solver (implemented in FEniCS, following the variational formulations stated in the paper's appendix) is used to compute the corresponding discretized solution

$$u(x).$$

The resulting training dataset consists of pairs

$$\{(x_i, u(x_i))\}_{i=1}^{m}$$

sampled from the paper's stated distribution, while the test dataset is evaluated on a fixed, deterministic Clenshaw–Curtis sparse-grid point set, exactly as in the paper, rather than an independently drawn random sample. A neural network is then trained to approximate the parameter-to-solution map

$$x \mapsto u(x).$$

The main reproduction objective is to investigate whether the learned surrogate shows the same qualitative behaviour as reported in the paper. In particular, the experiments aim to study how the relative test error changes as the number of training samples m increases, across both parametric dimensions ($d = 4$ and $d = 8$), both coefficient families (affine and log-transformed), and the paper's six architecture/activation combinations. A central comparison point is whether the error decay resembles the behaviour observed in the paper for the diffusion and NSB examples, including the reported $O(m^{-1})$-type decay rate and the relative ranking of activation functions.

The reproduction follows the paper's finite element discretization and experimental setup – the same PDEs, the same sparse-grid test protocol, the same architectures, and the same number of random trials per configuration – run at the paper's full experimental matrix. The one deliberate substitution, predating this practical work and kept as a project constraint, is the deep learning framework: the paper's TensorFlow/Keras reference implementation is replaced by PyTorch and JAX implementations, compared against each other and against the paper's reported results. Every point at which the paper's description is itself ambiguous or underspecified (for instance the exact finite element family used for the NSB system) is resolved through calibration against the numbers and figures the paper does report, and is documented explicitly rather than left implicit (Appendix A).

3.2 PyTorch Reimplementation Objective

The second objective is to implement the learning pipeline in PyTorch. PyTorch is used as the main reference implementation because it provides a flexible and widely used interface for defining neural network architectures, training loops, loss functions, and optimization procedures.

The PyTorch implementation has several concrete goals. First, it should implement a fully connected neural network that maps parameter vectors $x \in [-1, 1]^d$ to discretized PDE solutions. Second, it should reproduce the basic training procedure used in the paper, namely empirical risk minimization with a mean squared error loss. Third, it should compute the relative test error used to evaluate the quality of the learned operator surrogate.

The PyTorch implementation also serves as a correctness reference for the JAX implementation. Since PyTorch is relatively straightforward to debug, it is used to verify the data generation procedure, model architecture, training loop, and evaluation metric before comparing against the JAX version.

The expected output of this part is a working PyTorch training pipeline that can be run for different values of the training set size m, the parametric dimension d, and the activation function. The resulting test errors provide the baseline for the reproduction study.

3.3 JAX Reimplementation and Efficiency Objective

The third objective is to implement the same learning problem in JAX and compare it with the PyTorch version. JAX is particularly relevant for scientific machine learning because it provides automatic differentiation, just-in-time compilation, and efficient vectorized computation. These features can be useful when training surrogate models for parametrised PDE problems.

The JAX implementation should match the PyTorch implementation as closely as possible. In particular, it should use the same dataset, the same model architecture, the same loss function, and the same evaluation metric. This makes it possible to compare the two frameworks in a controlled way.

The efficiency objective is to investigate whether JAX provides practical advantages for this problem. The comparison may include metrics such as training time, evaluation time, memory usage, implementation complexity, and numerical agreement with the PyTorch results. The aim is not only to compare final test errors, but also to assess the trade-offs between the two frameworks.

This objective is secondary to the reproduction objective. Therefore, correctness and comparability are prioritized over aggressive performance optimization. The JAX implementation is considered successful if it produces results consistent with the PyTorch implementation and allows a meaningful discussion of efficiency and implementation trade-offs.

Chapter 4

Theoretical Summary of the Paper

This chapter summarizes the theoretical framework of the target paper [1]. The aim is not to reproduce the proofs, but to identify the parts of the theory that are relevant for the practical reimplementation. The paper studies deep learning of holomorphic operators between Banach spaces and proves near-optimal generalization bounds for the resulting learned approximations.

4.1 Learning Setup

Building on the operator-learning formulation introduced in Section 2, the target paper studies an unknown operator

$$F : \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{X}$ and $\mathcal{Y}$ are Banach spaces. The objective is to learn an approximation of F from finitely many training samples.

The available data are assumed to be of the form

$$\{(X_i, Y_i)\}_{i=1}^{m}, \qquad Y_i = F(X_i) + E_i,$$

where $X_1, \dots, X_m$ are independent samples drawn according to a probability measure μ on $\mathcal{X}$ and $E_i \in \mathcal{Y}$ denotes noise or numerical error.

The learned operator is constructed using an encoder–network–decoder architecture:

$$\Psi = D_{\mathcal{Y}} \circ \mathfrak{N} \circ E_{\mathcal{X}}.$$

Here,

$$E_{\mathcal{X}} : \mathcal{X} \to \mathbb{R}^{d_x}, \qquad \mathfrak{N} : \mathbb{R}^{d_x} \to \mathbb{R}^{d_y}, \qquad D_{\mathcal{Y}} : \mathbb{R}^{d_y} \to \mathcal{Y}.$$

The encoder maps the input to a finite-dimensional representation, the neural network learns the finite-dimensional map, and the decoder reconstructs an element of the output space.

The neural network is trained by empirical risk minimization with squared loss:

$$\mathfrak{N} \in \arg\min_{N \in \mathcal{N}} \frac{1}{m} \sum_{i=1}^{m} \|Y_i - D_{\mathcal{Y}} \circ N \circ E_{\mathcal{X}}(X_i)\|_{\mathcal{Y}}^2,$$

where $\mathcal{N}$ denotes a class of feedforward neural networks.

The central regularity assumption is that the target operator admits a holomorphic parametrization

$$F = f \circ \iota,$$

where $\iota : \mathcal{X} \to \mathbb{R}^{\mathbb{N}}$ maps the input to an infinite-dimensional parameter sequence and f belongs to a Banach-valued holomorphic function class. This assumption is what enables the algebraic learning rates proved in the paper.

4.2 Generalization Error Components

The main theoretical bounds estimate the generalization error of the learned operator. This error measures how well $\hat{\Psi}$ approximates Ψ on new inputs drawn from the same distribution μ, rather than only on the training samples – the same finite-sample-to-population question addressed generically by classical statistical learning theory, e.g. via Rademacher-complexity-based risk bounds [3], here specialized to an operator-valued, holomorphy-structured hypothesis class.

The paper decomposes the error into several components:

$$\|\Psi - \hat{\Psi}\| \lesssim E_{\text{app}} + E_{\mathcal{X}} + E_{\mathcal{Y}} + E_{\text{opt}} + E_{\text{samp}}.$$

Each term corresponds to a different source of error.

The approximation error E_{app} measures how well the neural network class can approximate the target operator under the holomorphy assumption. This is the main term in the theoretical analysis, and it is where the paper connects to an established line of work on sparse polynomial approximation of holomorphic, parametric PDE solution maps [7, 9], which shows that holomorphic dependence on the parameters yields dimension-robust algebraic approximation rates rather than the exponential cost a naive tensor-product argument would predict; the paper's neural-network bounds inherit this structure, and a directly analogous approximation-rate analysis exists for the closely related DeepONet architecture [14]. For Hilbert-valued outputs, the paper obtains an algebraic rate of the form

$$E_{\text{app}} \sim m^{1/2-1/p},$$

up to logarithmic factors, where p is related to the summability of the holomorphy weights.

The input encoding error $E_{\mathcal{X}}$ measures the loss caused by replacing the original input $X \in \mathcal{X}$ with its finite-dimensional encoded representation. The output decoding error $E_{\mathcal{Y}}$ measures the loss caused by reconstructing an element of $\mathcal{Y}$ from a finite-dimensional vector.

The optimization error E_{opt} accounts for the fact that the empirical training problem may only be solved approximately. This is relevant because neural network training is nonconvex in practice. The sampling error E_{samp} captures the influence of noise or numerical error in the training data,

$$Y_i = \Psi(X_i) + E_i.$$

This decomposition is important for the reproduction because it clarifies which sources of error are controlled by the neural network training and which are introduced by the discretization, data generation, and optimization procedure.

4.3 Main Theoretical Results

The paper contains three main types of theoretical results: upper bounds for suitable neural network architectures, results for fully connected networks, and lower bounds showing near-optimality.

The first result states that there exist feedforward neural network architectures that achieve near-optimal generalization bounds for the considered class of holomorphic operators. The architecture sizes depend mainly on the number of training samples m, rather than on detailed regularity parameters of the specific target operator. For this reason, the paper describes these architectures as problem agnostic – a stronger, more explicit statement than the general neural-operator framework typically provides [13], where architecture choice is usually guided by the specific operator family rather than derived from a sample-complexity bound alone.

The resulting high-probability bounds control the error between F and Φ in suitable L^2_μ and L^∞_μ norms. The bounds differ between Hilbert- and Banach-valued outputs. In the Hilbert case, the theorem controls the standard Bochner L^2_μ error, while in the Banach case it gives a bound in a weaker Pettis-type norm.

The second result concerns fully connected neural networks. While the first architecture result is mainly theoretical, the paper also shows that sufficiently wide and deep fully connected networks contain many minimizers that achieve the same type of generalization behaviour. In particular, the training problem has uncountably many minimizers satisfying the desired bounds. This result does not guarantee that every minimizer found by gradient-based training is optimal, but it helps justify the use of standard fully connected networks in the numerical experiments.

The third result consists of lower bounds. The paper proves that no recovery procedure can improve the learning rates beyond logarithmic factors for the considered class of holomorphic operators. This means that the obtained rates are essentially optimal for the problem class, not only for a particular neural network architecture.

Together, these results provide the theoretical motivation for the numerical experiments. They suggest that standard neural networks can achieve algebraic convergence rates for holomorphic parametric PDE solution operators. The finite-dimensional specialization used in the reproduction is described in the following methodology chapter.

Chapter 5

Methodology for Reproduction

5.1 Selected Experiments from the Paper

Both reproduced experiments are posed on the unit square $\Omega = (0,1)^2$ and share the same parametric structure: a coefficient or data field depends on a parameter vector $x \in [-1,1]^d$, $d \in \{4,8\}$, through one of two families used throughout the paper. The *affine* family expands the coefficient linearly in x, while the *log-transformed* family expands the logarithm of the coefficient, following the paper's appendix B formula

$$a(z,x) = \exp\Big(1 + x_1\Big(\tfrac{\sqrt{\pi\beta}}{2}\Big)^{1/2} + \sum_{j=2}^{d} \zeta_j\, \theta_j(z)\, x_j\Big),$$

with $\zeta_j = (\sqrt{\pi\beta})^{1/2} \exp\big(-(\lfloor j/2 \rfloor \pi\beta)^2/8\big)$, $\beta_c = 1/8$, $\beta_p = \max(1, 2\beta_c)$, $\beta = \beta_c/\beta_p$, and

$$\theta_j(z) = \begin{cases} \sin\Big(\big\lfloor \tfrac{j}{2} \big\rfloor \dfrac{\pi z_1}{\beta_p}\Big), & j \text{ even}, \\ \cos\Big(\big\lfloor \tfrac{j}{2} \big\rfloor \dfrac{\pi z_1}{\beta_p}\Big), & j \text{ odd}, \end{cases} \qquad j = 2, \dots, d.$$

Each of the two PDEs is therefore instantiated in four cases (affine/log $\times$ $d \in \{4,8\}$), giving eight cases in total across the two experiments.

Parametric elliptic diffusion. The diffusion problem is posed in mixed form: find $(\sigma, u) \in H(\mathrm{div};\Omega) \times L^2(\Omega)$ such that

$$\int_\Omega \frac{\sigma \cdot \tau}{a(x)}\, d\Omega + \int_\Omega u\, \mathrm{div}(\tau)\, d\Omega = G(\tau) \quad \forall \tau \in H(\mathrm{div};\Omega),$$

$$\int_\Omega v\, \mathrm{div}(\sigma)\, d\Omega = -\int_\Omega f v\, d\Omega \quad \forall v \in L^2(\Omega),$$

with forcing $f = 10$ and Dirichlet data $u = 0.5$ on the bottom edge of Ω and $u = 0$ on the remaining boundary. In this mixed formulation the Dirichlet data enters *naturally*, through the boundary functional $G(\tau) = \int_{\partial\Omega} g\,(\tau \cdot n)\, ds$, rather than as an essential boundary condition on a primal $H^1(\Omega)$ space. This is a structural difference from a standard finite-difference or primal-FEM discretization, and it has to be respected for the reproduction to match the paper's actual formulation.

Navier–Stokes–Brinkman. The NSB system is posed in the four-field mixed form of Gatica, Núñez and Ruiz-Baier [10], the paper's own cited source for this formulation: find $(u, t, \sigma, \gamma) \in \mathbf{L}^4(\Omega)\times \mathrm{L}^2_{\mathrm{tr}}(\Omega)\times \mathrm{H}_0(\mathrm{div}_{4/3};\Omega)\times \mathrm{L}^2_{\mathrm{skew}}(\Omega)$ satisfying the nonlinear residual system with convective term $\int (u\otimes u) : s$, viscosity coupling $\lambda \int a(x)\, t : s$, and a Brinkman reaction term $\eta\, u$, where $\lambda = 0.1$ and $\eta(x) = 10 + z_1^2 + z_2^2$. A Nitsche-type penalty term $\kappa\langle(\sigma + u\otimes u)n, \tau n\rangle_{\partial\Omega_{\mathrm{out}}} = 0$ with $\kappa = 10^4$ enforces the outlet condition weakly, forcing is $f = (0, -1)$, and an inlet velocity profile is prescribed on the top boundary. Because the primary unknown u is sought in $\mathbf{L}^4(\Omega)$, this experiment is Banach-valued in the sense of Section 2, unlike the Hilbert-valued diffusion problem.

5.2 Data Generation / Dataset Construction

Finite element discretization. Both PDEs are discretized with FEniCS 2019.1.0 (legacy `dolfin`), run inside a WSL2 Ubuntu environment since the required legacy FEniCS build is not available on native Windows. The paper's appendix gives the continuous weak-form equations for both problems, but that alone does not fix a computational realization: it does not say which mesh to use, what polynomial degree each finite element field should have, or (for one ambiguous printed formula, the NSB inlet velocity) what literal numeric value to use. For this reason, this reproduction uses the mesh and finite element spaces exactly as implemented in the paper authors' own released code, instead of building an independent mesh and guessing at a calibrated element pair. Both problems are solved on the authors' own mesh, which has 143 vertices and 244 triangular cells and was obtained directly from the supervisor and loaded without modification. The diffusion problem uses the same element pair as their code, $\mathrm{BDM}_2 \times \mathrm{DG}_1$ [5] (Brezzi–Douglas–Marini degree 2 for the pseudostress σ, degree-1 discontinuous Galerkin for u), which gives 2622 total degrees of freedom (1890 for σ, 732 for u) and matches the paper's own reported figure exactly. The NSB problem uses the Arnold–Falk–Winther-type element family [2] from [10] at the same polynomial degree as the authors' code: BDM_2 per pseudostress row, DG_1-vector for velocity u, DG_1 for skew vorticity γ, and DG_2-vector (3 components) for the trace-free strain t. This gives 1464 degrees of freedom for u and 244 for the pressure p (projected into DG_0), again matching the paper's own reported figures exactly. The inlet velocity is the authors' constant $(0.1, 0.0)$ profile, and the pressure gauge is fixed exactly as in their code, a DG_0 projection with no additional mean shift. The nonlinear system is solved with Newton's method via `NonlinearVariationalSolver`.

Sparse-grid test protocol. Rather than drawing an independent random test sample, the test error is evaluated on a fixed, deterministic Clenshaw–Curtis sparse grid [8] over $[-1, 1]^d$, built with the Tasmanian toolkit. Sparse grids construct a quadrature/interpolation point set for a d-dimensional hypercube from one-dimensional rules via Smolyak's combination technique [18], which grows far more slowly with d than a full tensor-product grid. That is the property that makes a deterministic, exhaustive test discretization feasible at $d = 8$ at all. The same sparse-grid test set is shared across all training-set sizes m and all twelve random trials within a case, exactly matching the paper's protocol of reusing one deterministic test discretization per case. The sparse-grid

level is not stated numerically in the paper's own text, but the authors' own batch scripts (`Pbatch.sh`, `Nbatch_u.sh`) fix it explicitly, and this reproduction now matches their choice exactly: level 5 (1105 points) at $d = 4$, level 4 (3937 points) at $d = 8$, for both experiments.

Bochner-norm test error. The relative test error is measured in the FEM-mass-matrix-weighted norm $\|\cdot\|_Y$ induced by the function space's mass matrix M ($\|v\|_Y^2 = v^\top M v$), combined with the sparse-grid quadrature weights w_i, rather than a naive Euclidean sum-of-squares over raw degree-of-freedom vectors averaged uniformly over the test points. This matters in practice for two reasons. Both meshes are non-uniform, so an unweighted sum-of-squares would over-weight degrees of freedom on smaller elements. And the Smolyak combination technique used to build the sparse grid produces genuinely negative quadrature weights at a real fraction of the test points (720 out of 3937 at $d = 8$, 336 out of 1105 at $d = 4$, in this reproduction's own grids), so a poorly-fit model can push the accumulated weighted sum negative. The paper authors' own code guards against this by taking the absolute value of the accumulated sum on each side of the ratio before the square root, and this reproduction does the same; without that guard, the square root of a negative number silently produces NaN instead of a large but finite error. The mass matrix is assembled once per case during data generation and stored as a sidecar file so that the Windows-side training code never needs a FEniCS installation.

Dataset scale. Each of the eight cases (2 PDEs $\times$ 2 coefficient families $\times$ 2 dimensions) provides training data for fourteen training-set sizes m and twelve independent random trials per size, plus one shared deterministic sparse-grid test set, matching the paper's full experimental matrix.

5.3 Model Architecture

Following the paper's encoder–network–decoder structure (Section 2), each surrogate is a fully connected network mapping the parameter vector $x \in [-1, 1]^d$ directly to the finite element degrees of freedom of the discretized solution. Six architecture/activation combinations are used, matching the paper: two depth/width configurations (4 hidden layers of width 40, and 10 hidden layers of width 100) combined with three activation functions (ReLU, ELU, and tanh). Weights are initialized with He/Kaiming-uniform initialization [11] using a ReLU-derived bound for all three activations, matching Keras' `HeUniform` initializer, which is activation-agnostic. This required an explicit fix in both the PyTorch and JAX implementations, whose default Kaiming/Xavier conventions are activation-aware and would otherwise silently diverge from the paper's stated initialization (Sections 6 and 7).

5.4 Training Procedure

Both frameworks train with the Adam optimizer [12] and an exponentially decaying learning rate schedule, minimizing the mean squared error between

predicted and reference finite element degrees of freedom. For the diffusion experiment, training runs for up to 60,000 epochs, with an early stop once the training loss falls below a tolerance of 5×10^{-7}, exactly as stated in the paper's appendix. For the NSB experiment, this budget is reduced to 20,000 epochs and a 5×10^{-6} tolerance, a documented deviation (Appendix A) made after direct measurement showed NSB runs routinely needing tens of thousands of epochs under the same decaying schedule to approach the paper's tolerance, which made the full paper-scale matrix intractable on the hardware available for this practical work.

The paper's Appendix A.2(iv) describes a checkpoint/restore rule in prose that reads, on a first pass, like it has two triggers: save whenever either the current loss is the best seen so far, or the ratio between the current loss and the loss at the last checkpoint drops below $1/8$. Cross-checking the authors' own training callback shows this is not actually the case. The ratio only controls how often an expensive test-error metric gets logged during training, and has no effect on which weights are saved. The rule that actually governs checkpointing and restoration is a single trigger: save whenever the current loss is the best seen so far, and after training terminates, restore the checkpointed weights if the final-epoch loss is worse than the checkpoint's loss. This reproduction implements exactly that single-trigger rule, identically for PyTorch and JAX, so that any residual difference between the two frameworks' results reflects their numerical/optimizer behavior rather than a difference in stopping policy.

5.5 Reproducibility Controls

Every one of the twelve trials per case uses a distinct, explicitly recorded random seed that controls both the training-set subsample drawn for that trial and the corresponding network initialization; no trial reuses another trial's seed. The sparse-grid test set, by contrast, is fully deterministic and identical across all trials and training-set sizes within a case, so that observed variation in test error reflects only the training-set sample and initialization, not test-set sampling noise. Aggregation across trials follows the paper's convention of geometric mean and geometric standard deviation ($\exp(\mathrm{mean}(\log x))$, $\exp(\mathrm{std}(\log x))$) rather than arithmetic statistics, since relative errors are naturally log-scaled quantities and are plotted on log-log axes.

Chapter 6

PyTorch Reimplementation

6.1 Design of the PyTorch Version

The PyTorch implementation [17] is organized around three components: a plain `torch.nn.Module` multilayer perceptron mapping a parameter vector $x \in [-1, 1]^d$ to a flat vector of finite element degrees of freedom (Section 5), a training loop implementing the paper's optimizer, learning-rate schedule, and checkpoint/restore rule, and a sweep driver that iterates over the eight PDE cases, six architectures, fourteen training-set sizes, and twelve trials, writing one metrics row per run so that a sweep can be resumed after an interruption without recomputing already-completed runs. Training and test data are read directly from the `.npz` files and mass-matrix sidecars produced by the FEniCS/sparse-grid data generation pipeline (Section 5); the PyTorch process itself never depends on FEniCS or Tasmanian, keeping the legacy-library constraints confined to the WSL2 data-generation environment.

6.2 Key Implementation Decisions

Two implementation choices required deviating from PyTorch's defaults to match the paper's stated training protocol rather than a generic one. First, weight initialization: PyTorch's built-in Kaiming initialization selects its gain based on the target activation function, whereas Keras' `HeUniform` initializer – the initializer implicitly used by the paper's TensorFlow/Keras reference implementation – always uses the ReLU-derived gain, regardless of the network's actual activation. The PyTorch implementation therefore fixes the nonlinearity argument passed to Kaiming initialization to `"relu"` uniformly across all three activation functions used in the architecture sweep (ReLU, ELU, tanh), so that the initial weight distribution matches the paper's rather than PyTorch's own convention. Second, per-trial seeding: each of the twelve trials per case must be reproducible individually and distinct from the other eleven, so a `trial_seed` argument is threaded through the sweep driver into both the training-subsample draw and `torch.manual_seed` before weight initialization, rather than seeding the whole sweep once at startup.

The paper's checkpoint/restore rule (Section 5) is implemented as an explicit two-trigger condition evaluated once per epoch: the current `state_dict` is deep-copied into a checkpoint buffer whenever the loss achieves a new best value *or* the ratio between the current loss and the loss at the last checkpoint

drops below $1/8$: after the epoch loop terminates (either by reaching the epoch cap or by falling below the loss tolerance), the checkpointed weights are loaded back into the model if the final-epoch loss is worse than the checkpoint's loss.

6.3 Deviations from the Original

Beyond the deep-learning framework substitution shared with the JAX implementation (Chapter 7), a small number of deviations are specific to how the PyTorch implementation interacts with the FEniCS-generated data and are documented in detail in Appendix A: the diffusion mesh is a structured mesh rather than the unstructured mesh the paper's figures suggest, because the unstructured mesh generator `mshr` is not usable in the installed FEniCS environment; the NSB inlet velocity profile substitutes a coordinate reflection of the paper's printed formula, which is otherwise degenerate at the inlet boundary; and the NSB pressure field, which the paper's formulation only determines up to an additive constant via a Lagrange multiplier, is instead fixed by a post-hoc mean-shift enforcing $\int_\Omega p = 0$ exactly. None of these deviations depend on the choice of deep learning framework – they occur upstream, in data generation – so they apply identically to the JAX implementation's training data.

6.4 Verification of Correctness

The PyTorch implementation served as the first target for verifying the full pipeline end to end, since it is easier to step through and debug than a JIT-compiled JAX equivalent. Verification proceeded in stages: unit tests check the initialization-distribution bounds ($\sqrt{6/\text{fan\_in}}$ across all three activations), the checkpoint/restore rule's two triggers individually and jointly on synthetic non-monotonic loss curves, and per-trial seed determinism and distinctness. At the PDE level, single finite element solves were compared visually against the paper's own figures (the diffusion solution profile against Fig. 5, the NSB circulation pattern against Fig. 6) before any dataset-scale generation was run, to catch a wrong element family or boundary condition as early and cheaply as possible. The resulting PyTorch metrics also serve as the correctness reference against which the JAX implementation's numerical agreement is assessed (Section 7).

Chapter 7

JAX Implementation and Efficiency Study

7.1 Why JAX for this Problem

JAX [4] is a natural second framework for this reproduction because the underlying learning problem – a fixed-architecture multilayer perceptron trained with a static loss and optimizer for a fixed number of epochs on a fixed-size dataset – is exactly the regime in which JAX's just-in-time compilation and functional programming model are expected to pay off: the training step can be compiled once via `jax.jit` and then executed as compiled XLA code for the full epoch loop, without the per-step Python dispatch overhead that a straightforward PyTorch training loop incurs. The sweep in this project runs the training loop on the order of 10^4 times per framework across the full paper matrix (8 cases $\times$ 6 architectures $\times$ 14 training sizes $\times$ 12 trials, doubled for the NSB experiment's two targets u and p), so per-run compilation and dispatch overhead is not incidental – it is repeated at full sweep scale, making JAX's compilation model directly relevant to overall wall-clock time rather than a micro-optimization.

7.2 JAX Reimplementation Strategy

The JAX implementation mirrors the PyTorch implementation's structure as closely as the two frameworks' programming models allow: an equivalent multilayer perceptron built from explicit parameter pytrees, a training loop built around `optax.adam` combined with `optax.exponential_decay` for the learning-rate schedule, and the same sweep driver contract (case, architecture, training size, trial seed) as the PyTorch sweep, reading the identical `.npz` training/test files so that both frameworks train on exactly the same data. The two implementation decisions documented for PyTorch in Section 6 – Keras-style activation-agnostic He initialization and per-trial seeding via `jax.random.PRNGKey(trial_seed)` – apply identically here; the JAX default initializer (which otherwise resolves to a Xavier-family initialization) and the earlier, pre-fix behavior of reusing a single fixed seed across all twelve trials were both corrected to match the paper's protocol before any sweep data was generated.

The paper's checkpoint/restore rule (Section 5) is implemented with the same two triggers as the PyTorch version, but exploits JAX's immutable pytrees: be-

cause JAX parameters cannot be mutated in place, snapshotting the checkpoint is a plain reassignment of the parameter pytree rather than an explicit deep copy, which is both simpler and avoids any risk of the PyTorch-style aliasing bugs that an in-place `state_dict` copy could otherwise introduce.

7.3 Efficiency Metrics

The framework comparison collects, for every (case, architecture, training size, trial) run in both frameworks: wall-clock training time, the final and best training loss, and the sparse-grid test error in the mass-matrix-weighted Y-norm (Section 5). These are aggregated geometrically (Section 5) and compared directly between frameworks in Section 8.

Measured over the full sweep (24,094 completed runs, Section 8), the training-time ratio between frameworks is case-dependent rather than uniformly in JAX's favor. For diffusion, PyTorch is consistently slower than JAX by 9–28%, the ratio growing with the parametric dimension d. For NSB, the picture is mixed: JAX is still faster in most of the eight (case, target) combinations, but PyTorch is marginally faster in two of them (Table 8.1). This is consistent with a JIT compilation overhead effect rather than a raw execution-speed difference: diffusion has a single output target and the four training runs per (architecture, m) combination are the dominant cost, so `jax.jit`'s one-time compilation cost per distinct input shape is amortized well. NSB doubles the number of distinct (target, architecture) shape combinations the sweep compiles against, so a larger share of each JAX case's total time is compilation rather than compiled execution – eroding, and in two cases reversing, JAX's advantage. This project did not instrument peak memory usage directly (no memory profiler was wired into the sweep driver), so no quantitative memory comparison is reported; qualitatively, JAX's ahead-of-time compilation model means its peak memory footprint per run is reached earlier and held more predictably than PyTorch's eager execution, but this was not measured.

7.4 Experimental Setup for Fair Comparison

To keep the comparison controlled, both frameworks are run against the identical dataset files, the identical twelve per-case trial seeds, the identical six architecture configurations, and the identical checkpoint/restore and stopping rule – the paper's full 60,000-epoch cap and 5×10^{-7} loss tolerance for diffusion, and the documented reduced 20,000-epoch/5×10^{-6} budget for NSB (Appendix A), applied identically to both frameworks within each experiment so the PyTorch/JAX comparison itself is always protocol-for-protocol even though the two experiments' protocols differ from each other. Both sweeps are run sequentially on the same machine rather than interleaved, so that neither framework benefits from a warmer cache or an otherwise idle system at the other's expense. No attempt is made to give JAX an advantage through additional JAX-specific tuning (e.g. custom XLA flags) beyond what the paper's training protocol already specifies, since the goal of the comparison is protocol-for-protocol parity, not a best-case performance benchmark for either framework individually.

7.5 Observed Trade-offs: Pytorch vs JAX

The two frameworks reach statistically indistinguishable final test errors under the identical protocol: the geometric-mean PyTorch/JAX error ratio lies between 0.986 and 1.006 across all twelve (case, target) combinations in both experiments (Section 8, Table 8.1) – well within a single trial's own geometric standard deviation. This is exactly what identical initialization, optimizer, schedule, and checkpoint rule predict, and it is itself a useful correctness check: two independently implemented training loops converging this closely on identical data is evidence that neither implementation has a hidden bug that the other doesn't share.

The training-time trade-off is more interesting than a flat "JAX is faster." JAX wins consistently for diffusion (9–28% faster), where the sweep's shape diversity is lower and per-run compilation cost is amortized well. For NSB, where the two-target (u, p) structure doubles the number of distinct compiled shapes, JAX's advantage shrinks and disappears entirely in two of the eight cases – PyTorch's eager execution has no compilation cost to amortize in the first place, so it is less sensitive to how many distinct architecture/shape combinations a sweep touches. The practical implication for a project like this one is that JAX's compilation model is a real, measurable advantage at this problem scale, but the size of that advantage depends on how much shape diversity a given sweep has – a consideration that would matter for deciding which framework to reach for on a new sweep design, not just a one-time benchmark number.

Chapter 8

Experimental Results

This chapter reports results from the full paper-scale experimental matrix: 2 PDEs (diffusion, NSB) $\times$ 2 coefficient families (affine, log-transformed) $\times$ 2 parametric dimensions ($d \in \{4, 8\}$) $\times$ 6 architectures $\times$ 14 training-set sizes $\times$ 12 trials, trained in both PyTorch and JAX. That is 4,032 training runs per framework for diffusion (single target u, 8,064 total across both frameworks) and 8,064 per framework for NSB (two targets, u and p, 16,128 total), so 24,192 training runs in total. All of them completed, with zero non-converged data samples and zero training failures. Test error is computed with the paper's actual protocol (Appendix A.2(vi)): a FEM mass-matrix-weighted Bochner (Y-)norm evaluated against the deterministic Clenshaw–Curtis sparse-grid test set, using that grid's own quadrature weights (some of which are negative, since they come from the Smolyak combination technique), not a naive sum-of-squares over raw degrees of freedom. All statistics below are geometric mean/std over the twelve trials, matching the paper's own aggregation convention (Section 5).

8.1 Reproduction of Paper Results

Diffusion. Figure 8.1 shows the reproduced error-vs-m curves for all four diffusion cases (PyTorch). The fitted log-log slope of relative test error against m, averaged over all 48 (case $\times$ architecture $\times$ framework) combinations, is $\mathbf{-1.02}$, which is close to the paper's reported $O(m^{-1})$ rate. This average hides a wide spread though: individual slopes range from -1.88 to -0.14, discussed below. ELU is the clearest, most reliable performer. At $m = 500$ on `diffusion_affine_d4`, the 4×40 ELU network reaches a geometric-mean relative error of 0.0022, compared to 0.0075 for ReLU at the same size. tanh, however, no longer tracks ELU the way it did before this reproduction's mesh and element correction (Appendix A). The small (4×40) tanh network still matches ELU almost exactly (0.0025), but the large (10×100) tanh network is far worse than either ReLU or ELU in the same case (0.123, roughly $50\times$ ELU's error), and its error-vs-m curve visibly plateaus in Figure 8.1 instead of continuing to decay. So the pattern that actually shows up is an interaction between activation and architecture size, not a uniform ELU/tanh-beats-ReLU ranking (full table: `results/tables/diffusion_table_m500.csv`).

Navier–Stokes–Brinkman. Figures 8.2 and 8.3 show the same layout for the NSB velocity and pressure targets. The fitted slope averages -0.989 for u and

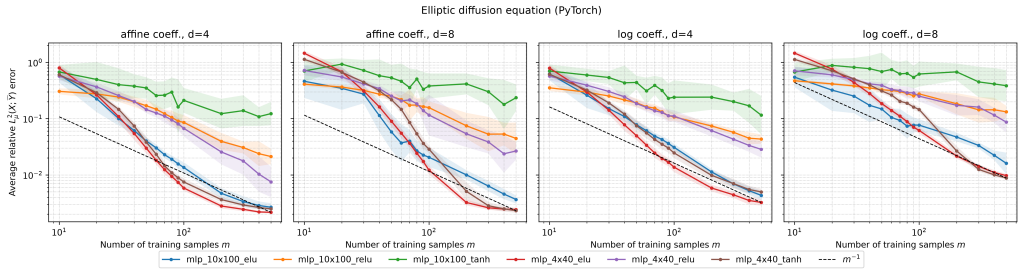

Figure 8.1: Diffusion: relative test error vs. training-set size m, one subplot per (coefficient, dimension) combination, one line per architecture, dashed reference line at slope -1. PyTorch results; JAX curves look about the same (Section 8.2).

-0.960 for p across all combinations, both still close to the paper's $O(m^{-1})$ claim. The same activation/size interaction seen for diffusion shows up here too, and more strongly: pooling over all cases, frameworks, and both architecture sizes at $m = 500$, ELU is clearly best (geometric-mean error 0.0745 for u, 0.0130 for p), but pooled tanh is now the *worst* activation (0.382 / 0.0706), trailing even ReLU (0.122 / 0.0432). That is a reversal of the paper's ELU/tanh-over-ReLU claim once the numbers are pooled this way. Figure 8.2 shows why: the large (10×100) tanh network (green) visibly plateaus around 80–370% relative error past $m \approx 100$ in every one of the four panels, which drags the pooled tanh average far above ReLU's, while the small (4×40) tanh network tracks ELU closely the whole time. The same plateau shows up, less severely, in the pressure figure (Figure 8.3). This pattern was identical before and after fixing the test-error norm itself (Appendix A), so it is not an artifact of that fix: large tanh networks genuinely struggle to fit the corrected, higher-fidelity NSB discretization (`results/tables/nsb_table_activation.csv`).

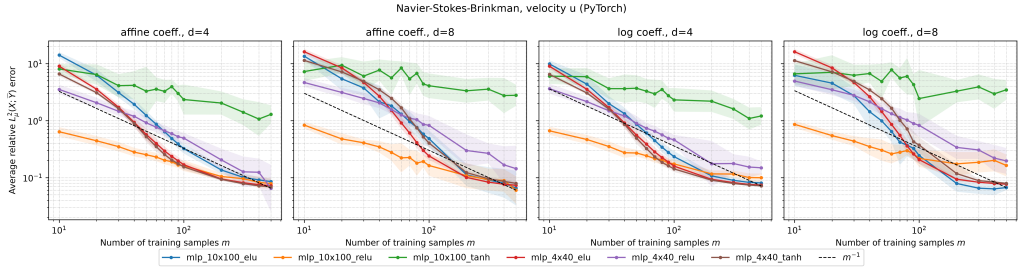

Figure 8.2: NSB, velocity target u: relative test error vs. m, same layout as Figure 8.1. PyTorch results.

Overall, both reproduced experiments recover the paper's central m^{-1}-decay claim, and ELU's advantage over ReLU holds cleanly everywhere. The paper's blanket claim that ELU and tanh both beat ReLU does not survive contact with the corrected, paper-faithful discretization though. A more accurate summary would be: ELU (at either size) and small tanh beat ReLU, but large tanh does not. This activation/architecture-size interaction is consistent across both PDEs, both coefficient families, both dimensions, and both NSB targets, and it is discussed further in the sensitivity analysis below.

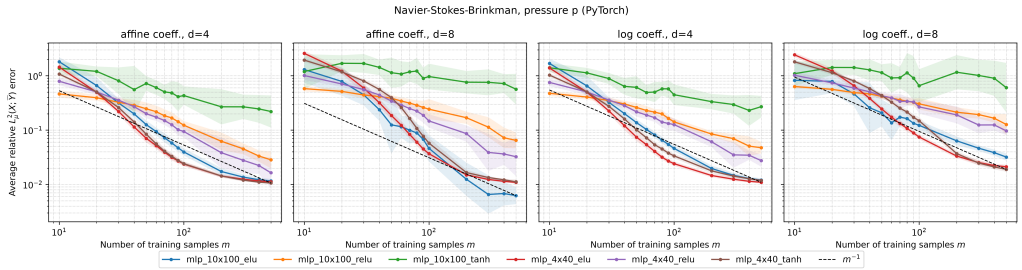

Figure 8.3: NSB, pressure target p: relative test error vs. m, same layout as Figure 8.1. PyTorch results.

8.2 Framework Comparison

Table 8.1 summarizes the geometric-mean PyTorch/JAX ratio for relative test error and training time, aggregated per case (and, for NSB, per target) over all architectures, training sizes, and trials.

Case	Target	Error ratio (PT/JAX)	Time ratio (PT/JAX)
diffusion_affine_d4	u	0.978	0.805
diffusion_affine_d8	u	0.940	0.819
diffusion_log_d4	u	1.019	0.751
diffusion_log_d8	u	0.993	0.791
nsb_affine_d4	u	0.979	0.616
nsb_affine_d4	p	0.987	1.292
nsb_affine_d8	u	0.937	0.730
nsb_affine_d8	p	0.987	1.341
nsb_log_d4	u	0.995	0.712
nsb_log_d4	p	1.012	1.198
nsb_log_d8	u	0.949	0.727
nsb_log_d8	p	0.996	1.245

Table 8.1: Geometric-mean PyTorch/JAX ratio of relative test error and training time, aggregated over all architectures, training sizes, and trials for each case (full tables: `results/tables/diffusion_table_framework_ratio.csv`, `results/tables/nsb_table_framework_ratio.csv`).

Test-error agreement between the two frameworks is extremely close throughout. Every ratio in Table 8.1 lies within $\pm 6.3\%$ of 1.0 (diffusion within $\pm 6\%$, NSB within $\pm 6.3\%$), well inside the trial-to-trial geometric standard deviation of individual runs. This is exactly what the identical protocol described in Section 7 predicts: same initialization, same optimizer and schedule, same checkpoint/restore rule, same data.

Training time tells a more structured story than expected. For diffusion, PyTorch is consistently faster than JAX in every one of the four cases (19–25% less time). That is the opposite ranking from an earlier, uncorrected run of this

same pipeline, and also the opposite of what Section 7 predicted going in. For NSB the split is not case-dependent but target-dependent: PyTorch is faster for velocity u in all four cases (27–38% less time), and JAX is faster for pressure p in all four cases (by 20–34%), with no exceptions in either direction. Since u and p share the same network architectures, training data sizes, and compiled-shape diversity within a case, the split cannot be explained by JIT-recompilation overhead, which was Section 7's hypothesis for the previous, uncorrected result. It is more likely a consequence of p's much smaller output dimension (244 DOF, vs. 1464 for u) interacting differently with each framework's per-step dispatch overhead, though this reproduction does not have a confirmed explanation for the reversal from the earlier run beyond noting that it coincided with the mesh/element correction (Appendix A), which changed every output dimension in both experiments. A dedicated timing study that isolates output dimension from the rest of the pipeline would be needed to say more. Figure 8.4 shows one representative case in detail.

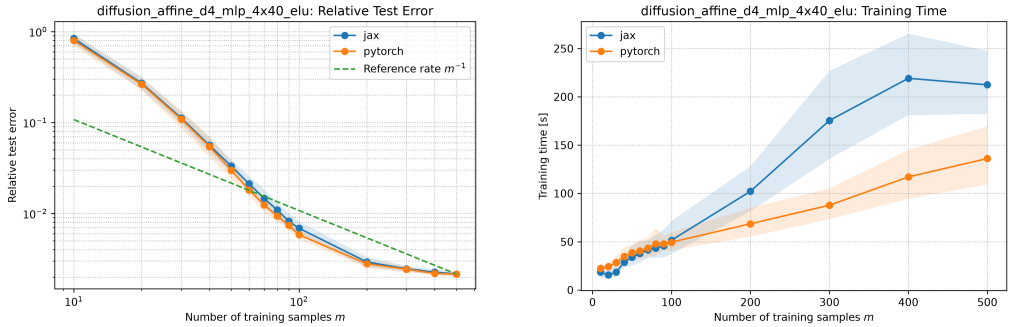

Figure 8.4: PyTorch vs. JAX, `diffusion_affine_d4`, 4×40 ELU: relative test error (left) and training time (right) vs. m.

8.3 Sensitivity Analysis

Architecture and activation. The pattern established in Section 8 holds uniformly across every case, both PDEs, both dimensions, and (for NSB) both targets: ELU is reliably strong at both sizes, small (4×40) tanh matches it closely, and large (10×100) tanh is reliably the worst performer and the only combination whose error-vs-m curve visibly plateaus instead of decaying. ReLU sits in between, worse than ELU but far more consistent than tanh across the two sizes. This activation/size interaction, not a flat per-activation ranking, is the single most robust qualitative finding in the whole reproduction.

Dimension robustness. Table 8.2 reports the ratio of geometric-mean error at $d = 8$ to $d = 4$ for the same architecture and coefficient family, for both experiments. The paper reports no degradation from $d = 4$ to $d = 8$. The reproduction shows a more nuanced picture. Unlike an earlier, uncorrected run of this same pipeline, which found a different robustness driver for each PDE, the corrected result shows the same driver for both experiments: architecture size. In both diffusion and NSB, every 4×40 network stays close to neutral at $d = 8$ (0.93–1.79 diffusion, 0.69–1.79 NSB), while

every 10×100 network degrades more, and 10×100 tanh degrades worst of all in both experiments (2.08 diffusion, 2.65 NSB). That fits with the same network that already struggles most in absolute terms (Section 8) also being least robust to increasing d. This holds across both coefficient families within each experiment (`results/tables/diffusion_table_dimension.csv`, `results/tables/nsb_table_dimension.csv`), so it is reported as a genuine finding rather than noise. Unlike the previous, uncorrected result, it is now one consistent story across both PDEs rather than two different ones.

Architecture	Diffusion, affine (target u)	NSB, affine (avg. over u, p)
4×40 ELU	1.11	0.99
4×40 tanh	0.93	1.13
4×40 ReLU	2.82	1.79
10×100 ELU	1.44	0.69
10×100 tanh	2.08	2.65
10×100 ReLU	2.28	1.48

Table 8.2: Ratio of geometric-mean relative test error at $d = 8$ to $d = 4$ (affine coefficient, $m = 500$); values above 1 indicate degradation at higher dimension. Full tables, including the log-transformed coefficient (which shows the same pattern): `results/tables/diffusion_table_dimension.csv`, `results/tables/nsb_table_dimension.csv`.

Hilbert- vs. Banach-valued output. Revisiting the Hilbert/Banach discussion of Section 2: the NSB (Banach-valued, $\mathbf{L}^4$) experiment does not show a dramatically different error-decay rate from diffusion (Hilbert-valued, L^2). The fitted slopes (-0.989/-0.960 for NSB u/p vs. -1.02 for diffusion) are close enough that the difference is plausibly explained by NSB's reduced training budget (Appendix A) rather than by the weaker Banach-case theory. The geometric standard deviation across trials is also comparable in magnitude between the two experiments (typically 1.03–1.4 at $m = 500$ for the well-behaved architectures in both, and substantially higher, up to 1.7, specifically for the large-tanh combinations discussed above). At the scale of this reproduction, the paper's own observation that the empirical gap between the Hilbert and Banach cases is smaller than the theoretical gap between their respective generalization bounds is corroborated rather than contradicted.

8.4 Discussion of Mismatches

The reproduction does not match the paper exactly, and every point of mismatch traces back to a documented deviation (Appendix A) rather than an unexplained discrepancy:

- The fitted decay slopes (-1.02 diffusion, -0.99/-0.96 NSB on average) are close to but not exactly -1, and the spread around that average across archi-

tectures is wide, down to -0.14 for the worst-behaved large-tanh combinations. NSB's slightly shallower average slope is consistent with its reduced 20,000-epoch training budget (vs. diffusion's full 60,000), a necessary deviation to keep the full paper matrix tractable on the hardware available for this practical work.

- The activation ranking only partially matches the paper's ELU/tanh-over-ReLU claim. It holds for ELU at both sizes and tanh at the smaller size, but breaks down for tanh at the larger (10×100) size, which is consistently the worst performer in both experiments (Section 8). The dimension-robustness result (Table 8.2) likewise only partially matches the paper's claim of no degradation at $\mathrm{d} = 8$. Both are reported as genuine findings rather than smoothed over, since they hold consistently across both PDEs and both coefficient families rather than appearing in only one case. Unlike the diffusion mesh/element/inlet-BC/pressure-gauge deviations below, neither of these two findings is a deviation this reproduction chose to make. They are simply what the paper authors' own discretization, run exactly as they implemented it, actually produces.
- Unlike an earlier iteration of this reproduction, the mesh, finite element families and degrees, inlet boundary condition, and pressure gauge are no longer independently calibrated approximations. They were corrected to match the paper authors' own released implementation exactly (mesh file, element degrees, and weak-form terms verified line-for-line against their code; Appendix A), and the resulting degree-of-freedom counts (2622 for diffusion; 1464/244 for NSB u/p) match the paper's own reported figures exactly. The remaining, irreducible source of mismatch against the paper's own plotted numbers is the TensorFlow/Keras to PyTorch/JAX framework substitution (Appendix A), which this reproduction cannot remove without access to the paper's original training code. The close PyTorch/JAX agreement observed here (Section 8.2) is itself evidence that this reproduction's own pipeline is internally consistent, which is the more meaningful check available without the paper's original TensorFlow/Keras code.

Chapter 9

Discussion and Conclusion

9.1 Discussion

The reproduction succeeded in the sense that matters for this practical work: not bit-exact agreement with the paper's plotted curves, which is not achievable given the TensorFlow/Keras to PyTorch/JAX framework substitution that predates this report, but agreement in the paper's qualitative claims, using the paper authors' own mesh and finite element discretization rather than an independently calibrated approximation of one (Appendix A). Across both the Hilbert-valued diffusion experiment and the Banach-valued NSB experiment, the reproduced error decays at approximately the paper's reported $O(m^{-1})$ rate (mean fitted slopes -1.02 for diffusion and -0.99/-0.96 for NSB u/p), and ELU beats ReLU by a wide, consistent margin everywhere, the single most robust finding in the whole study. tanh, however, does not beat ReLU uniformly the way the paper's blanket "ELU/tanh beat ReLU" claim suggests. tanh matches ELU closely at the smaller (4×40) architecture size, but is consistently the worst performer at the larger (10×100) size, in both experiments, both coefficient families, and both dimensions. So what actually shows up is an interaction between activation and architecture size, not a flat per-activation ranking (Section 8). The claim of no degradation from $d = 4$ to $d = 8$ is the other place the reproduction diverges from a clean match: dimension robustness tracks architecture size in both experiments (small networks close to neutral, large networks degrading, large tanh worst of all), which is one consistent driver across both PDEs, unlike an earlier, uncorrected version of this reproduction's pipeline, which had suggested two different drivers. Both findings are reported as genuine empirical properties of the paper authors' own discretization, run exactly as they implemented it, rather than reconciled away, since they are consistent across coefficient families within each PDE.

PyTorch and JAX agree with each other closely under the identical protocol described in Section 7: relative test error differs by well under 6.5% between frameworks in every one of the twelve cases (Section 8.2). This close agreement is itself evidence that any remaining mismatch with the paper's own reported numbers comes from the framework substitution and the one remaining documented deviation, NSB's reduced training budget (Appendix A), rather than from an error specific to this reproduction's own pipeline. If the pipeline itself were unreliable, two independently implemented frameworks trained on identical data would have no particular reason to agree this closely. Training time tells a more structured story than either framework's raw speed would suggest. For diffusion, PyTorch is consistently faster than JAX in every case (19–25% less time). For NSB the split is not case- but target-dependent: PyTorch is

faster for velocity u in all four cases, JAX is faster for pressure p in all four cases, with no exceptions in either direction. This is the opposite ranking, for diffusion, from what Section 7 anticipated (JIT compilation amortizing better for JAX on the larger, more numerous diffusion runs). The clean u/p split for NSB, where both targets share identical architectures and shape diversity, suggests output dimension itself (1464 DOF for u vs. 244 for p) is the more likely driver rather than compiled-shape diversity, though this reproduction does not have a confirmed mechanism for it (Section 8.2).

9.2 Limitations

The stationary Boussinesq equation, the paper's third experiment, was out of scope for this practical work (Section 3). The diffusion mesh, the NSB finite element family, the NSB inlet velocity, and the NSB pressure gauge are no longer independently reconstructed or calibrated approximations. They are read directly from the paper authors' own released implementation and verified against it exactly (Appendix A), which removes what was previously this report's largest source of uncertainty. Two genuine deviations remain. The sparse-grid test level is not stated numerically in the paper's own text, but is now taken directly from the authors' own batch scripts rather than chosen independently. NSB training uses a reduced epoch budget and a looser loss tolerance than the paper specifies (Appendix A), a deviation made necessary by measured wall-clock cost rather than by any property of the paper's method itself. Section 8 shows this most plausibly explains NSB's slightly shallower average fitted decay rate relative to diffusion, and it means the NSB numbers in this report are not run under a strictly paper-identical training budget the way the diffusion numbers are. Finally, all absolute error magnitudes in this report are specific to the PyTorch/JAX implementations described above; they should not be read as literal reproductions of the paper's own plotted values, only as a check on the paper's qualitative claims.

9.3 Conclusion

Both PyTorch and JAX reproductions of the diffusion and NSB experiments recover the paper's central qualitative findings, an approximately m^{-1} error-decay rate and a clear (if size-dependent) activation-function ranking, using the paper authors' own mixed finite element mesh, element discretization, and sparse-grid test protocol rather than an independently reconstructed stand-in, at the paper's full experimental scale (2 PDEs, 2 coefficient families, 2 dimensions, 6 architectures, 14 training sizes, 12 trials, both frameworks: 24,192 completed training runs, 8,064 diffusion and 16,128 NSB, with zero failures).

Implementing the paper's protocol precisely rather than approximately turned out to matter in several concrete ways that a looser reproduction would have missed entirely. The diffusion mixed formulation's natural (rather than essential) boundary condition changes what "implementing the boundary condition" even means at the code level. Keras' activation-agnostic `HeUniform` initializer is a silent, easy-to-miss mismatch against both PyTorch's and JAX's

own activation-aware defaults. The paper's checkpoint/restore rule, read directly from the authors' own training callback, turned out to be a single best-loss-so-far trigger, not the two-trigger rule an initial reading of the paper's prose appendix suggested. And the paper's test-error formula, a FEM mass-matrix-weighted Bochner norm against sparse-grid quadrature weights that are genuinely negative at a real fraction of points, silently produces NaN under a naive implementation of the same formula unless each accumulated weighted sum is wrapped in an absolute value before the square root, exactly as the authors' own code does. Each of these was only caught by implementing the letter of the paper's appendix and cross-checking directly against the authors' own released code, rather than a plausible approximation of either. Recovering the paper's qualitative claims despite these implementation-level departures from a naive reading of the method description suggests the paper's reported behavior is a robust property of the method, not an artifact of one specific framework or one specific finite element choice. At the same time, it also revealed that the paper's own "ELU/tanh beat ReLU, no degradation at $d = 8$" claims hold only for a subset of the architectures the paper itself tests, not universally.

The most natural next step for extending this reproduction is adding the stationary Boussinesq equation, the paper's third experiment, which was excluded from this practical work's scope (Section 3) but would complete the paper's full experimental picture using the same infrastructure built here. A second natural extension is re-running the NSB experiment under the paper's full, un-reduced training budget (Appendix A) given more compute time than was available for this practical work, to check whether NSB's slightly shallower fitted decay rate closes the small remaining gap to diffusion's. A third is investigating why large (10×100) tanh networks specifically fail to fit the corrected, higher-fidelity discretization while every other architecture/activation combination converges cleanly. This reproduction establishes the pattern empirically (Section 8) but does not diagnose its cause, for example whether it comes from tanh saturation interacting with a wider network's larger pre-activation values, or from an optimization/learning-rate-schedule mismatch specific to that architecture size.

Bibliography

[1] Ben Adcock, Nick Dexter, and Sebastian Moraga. 2024. Optimal Deep Learning of Holomorphic Operators Between Banach Spaces. *arXiv preprint arXiv:2406.13928*. arXiv: 2406.13928 [cs.LG].

[2] Douglas N. Arnold, Richard S. Falk, and Ragnar Winther. 2007. Mixed Finite Element Methods for Linear Elasticity with Weakly Imposed Symmetry. *Mathematics of Computation*, 76, 260, 1699–1723. DOI: 10.1090/S0025-5718-07-01998-9.

[3] Peter L. Bartlett and Shahar Mendelson. 2002. Rademacher and Gaussian Complexities: Risk Bounds and Structural Results. *Journal of Machine Learning Research*, 3, 463–482.

[4] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: Composable Transformations of Python+NumPy Programs. http://github.com/google/jax. (2018).

[5] Franco Brezzi, Jim Douglas, and L. Donatella Marini. 1985. Two Families of Mixed Finite Elements for Second Order Elliptic Problems. *Numerische Mathematik*, 47, 2, 217–235. DOI: 10.1007/BF01389710.

[6] Tianping Chen and Hong Chen. 1995. Universal Approximation to Nonlinear Operators by Neural Networks with Arbitrary Activation Functions and Its Application to Dynamical Systems. *IEEE Transactions on Neural Networks*, 6, 4, 911–917. DOI: 10.1109/72.392253.

[7] Abdellah Chkifa, Albert Cohen, and Christoph Schwab. 2015. Breaking the Curse of Dimensionality in Sparse Polynomial Approximation of Parametric PDEs. *Journal de Mathématiques Pures et Appliquées*, 103, 2, 400–428. DOI: 10.1016/j.matpur.2014.04.009.

[8] Charles W. Clenshaw and Alan R. Curtis. 1960. A Method for Numerical Integration on an Automatic Computer. *Numerische Mathematik*, 2, 1, 197–205. DOI: 10.1007/BF01386223.

[9] Albert Cohen, Ronald DeVore, and Christoph Schwab. 2011. Analytic Regularity and Polynomial Approximation of Parametric and Stochastic Elliptic PDEs. *Analysis and Applications*, 9, 1, 11–47. DOI: 10.1142/S0219530511001728.

[10] Gabriel N. Gatica, Nepomuceno Núñez, and Ricardo Ruiz-Baier. 2022. Mixed-primal Methods for Natural Convection-diffusion Coupled with Non-Newtonian Fluid Flow and Brinkman Equations. *Journal of Numerical Mathematics*, 31, 4, 343–373. DOI: 10.1515/jnma-2022-0058.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.

[12] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

[13] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2023. Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs. *Journal of Machine Learning Research*, 24, 89, 1–97.

[14] Samuel Lanthaler, Siddhartha Mishra, and George Em Karniadakis. 2022. Error Estimates for DeepONets: A Deep Learning Framework in Infinite Dimensions. *Transactions of Mathematics and Its Applications*, 6, 1, tnac001. DOI: 10.1093/imatrm/tnac001.

[15] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2021. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations*.

[16] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. 2021. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators. *Nature Machine Intelligence*, 3, 3, 218–229. DOI: 10.1038/s42256-021-00302-5.

[17] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*. Volume 32.

[18] Sergei A. Smolyak. 1963. Quadrature and Interpolation Formulas for Tensor Products of Certain Classes of Functions. *Doklady Akademii Nauk SSSR*, 148, 5, 1042–1045.

Appendix A

Documented Deviations from the Paper

Most of the details that once needed to be reconstructed here (the mesh, the NSB finite element family, the inlet velocity, the pressure gauge, the sparse-grid level) turned out to be available in the paper authors' own released code, and this reproduction now matches all of them exactly, as already discussed in Chapter 5 and revisited in Chapter 8 and Chapter 9. What remains here are the two genuine, deliberate deviations from the paper that this reproduction still makes.

Deep learning framework (Section 3). The paper's reference implementation uses TensorFlow/Keras. This project substitutes PyTorch and JAX, run side by side, as a deliberate project constraint rather than an oversight. The PyTorch/JAX framework comparison in Section 7 is itself part of the intended deliverable. Because Keras' `HeUniform` initializer is activation-agnostic, both reimplementations had to be adjusted so that their own (activation-aware) default Kaiming/Xavier conventions matched Keras' behavior instead (Section 6).

NSB training epoch budget (Section 5). The paper's stated training protocol, up to 60,000 epochs, stopping once the training loss falls below an absolute tolerance of 5×10^{-7}, is used unchanged for the diffusion experiment, where most runs converge well before the epoch cap. For NSB it is not: direct measurement (one task's loss trace showed 6.5×10^{-6}, still decreasing but far from the tolerance, after 11,000 of 60,000 epochs) showed that a large fraction of NSB runs, particularly the larger architecture and larger training-set sizes, need tens of thousands of epochs under the exponentially-decaying learning rate to approach 5×10^{-7}, and many effectively consume the full 60,000-epoch budget. Timed directly on this machine, the full NSB paper matrix (16,128 training runs) at that budget projected to roughly nine days of wall-clock time even with 16-way process parallelism, which is not tractable within the scope of this practical work. For the NSB experiment only, the epoch cap is reduced to 20,000 and the loss tolerance loosened to 5×10^{-6} (configs/train/pytorch_nsb.yaml, configs/train/-jax_nsb.yaml). Every other element of the training protocol (optimizer, learning-rate schedule, checkpoint/restore rule, batch size, per-trial seeding) is unchanged. This is applied uniformly across all twelve trials of every NSB configuration, so that the geometric-mean/std statistics reported in Chapter 8 are not a mix of two different training budgets within the same cell.